

顺时 × SEASONS

AI编程工具开发执行手册

如何用GLM/Cursor/Claude Code
高质量完成万亿级产品开发

提示词工程 · 分阶段执行 · 质量保证

81个精确提示词 · 150个代码文件

V1.0

顺时/SEASONS AI编程工具开发执行手册

如何用AI编程工具（GLM/Cursor/Claude Code）高质量完成项目

第一章：为什么直接丢大文档效果差

核心问题诊断

问题	原因	解决方案
AI输出代码不完整	文档太长，超出上下文窗口，AI“忘了”后半段	拆分为单任务提示词，每次只做一件事
代码质量低/敷衍	一次要求太多，AI只能“走马观花”实现	每个提示词只要求1个文件或1个模块
前后不一致	多次对话中AI不记得之前的设计决策	维护一份“项目记忆文件”，每次注入
风格不统一	没有明确的代码规范约束	在每次提示词开头注入代码规范
数据库/API不匹配	AI各写各的，没有统一的Schema参考	先确定Schema和API，后续严格引用
缺少错误处理	AI倾向写“happy path”	明确要求包含错误处理、边界情况
不能运行	缺少依赖/配置/导入	要求“完整可运行的文件”

黄金法则

- ✖ 错误做法：把6万字文档一次性丢给AI说“开发这个项目”

✔ 正确做法：拆成100+个精确的小任务，按依赖顺序逐步执行，每步验证

第二章：项目执行总策略

2.1 分层开发顺序

Phase 0: 项目骨架与规范 (Day 1)
↓
Phase 1: 数据库 + Models (Day 2-3)
↓
Phase 2: 核心Service层 (Day 4-6)
↓
Phase 3: API层 (Day 7-10)
↓
Phase 4: AI系统 (Day 11-13)
↓
Phase 5: Celery任务 (Day 14-15)
↓
Phase 6: 前端基础框架 (Day 16-18)
↓
Phase 7: 前端页面逐个开发 (Day 19-30)
↓
Phase 8: 集成测试 + 修复 (Day 31-35)
↓
Phase 9: 部署上线 (Day 36-40)

2.2 每次提示词的标准格式

每次给AI编程工具发送提示词时，严格使用以下格式：

```
# 任务：[具体任务名称]

## 项目上下文
[简短描述项目是什么，1-2句话]

## 技术栈
[列出本任务涉及的技术]

## 代码规范
[本项目的编码规范，保持每次一致]

## 已完成的相关文件
[列出本任务依赖的、已经完成的文件路径和关键内容摘要]

## 本次任务要求
[精确描述要创建什么文件、实现什么功能]

## 输出要求
1. 输出完整可运行的代码文件
2. 包含所有import语句
3. 包含完整的错误处理
4. 包含类型注解
5. 包含必要的注释（中文）
6. 不要省略任何代码，不要用"... "占位

## 验收标准
[列出这个任务完成后应该能做什么]
```

2.3 项目记忆文件（关键！）

创建一个 `PROJECT_CONTEXT.md` 文件，在每次对话开头注入。随着开发推进持续更新。

```
# 随时 项目记忆文件（每次对话必须先读取）

## 项目简介
顺时是一款AI养生生活陪伴系统，基于中医二十四节气+九种体质，为用户提供个性化养生方案。

## 技术栈
- 后端：Python 3.11 + FastAPI + SQLAlchemy 2.0 + Alembic + Celery + Redis
- 数据库：PostgreSQL 16 + pgvector + Meilisearch
- 前端：React Native 0.73 + Zustand + React Navigation 6
- AI：通义千问(主) + OpenAI(备)

## 代码规范
- Python：使用类型注解、Pydantic模型、async/await
- 命名：snake_case (Python), camelCase (TypeScript)
- API响应格式：{"code": 0, "message": "success", "data": {...}}
- 错误响应：{"code": "ERROR_CODE", "message": "中文友好消息", "status": 400}
- 所有时间使用UTC存储，前端转本地时区
- 所有ID使用UUID
- 数据库字段命名snake_case
- API路径使用kebab-case

## 数据库连接
DATABASE_URL=postgresql://postgres:password@localhost:5432/shunshi
REDIS_URL=redis://localhost:6379/0

## 目录结构（已确定）
backend/app/
├─ main.py
├─ config.py
├─ database.py
├─ models/           # SQLAlchemy模型
├─ schemas/          # Pydantic Schema
├─ api/v1/            # API路由
├─ services/          # 业务逻辑
├─ tasks/             # Celery任务
├─ core/              # 认证/依赖/异常
└─ knowledge/         # JSON知识库数据

## 已完成文件
[每完成一个文件就更新这个列表]
- [x] backend/app/config.py
- [x] backend/app/database.py
- [x] backend/app/models/user.py
- [ ] backend/app/models/solar_term.py
- ...

## 关键设计决策
1. 用户主键UUID，手机号唯一索引
2. 体质数据用JSONB存储评分
3. AI对话使用SSE流式响应
4. 推荐算法4维加权：体质40%+节气30%+时辰15%+历史15%
5. 会员验证中间件统一处理
```

第三章：Phase 0 — 项目骨架（精确提示词）

提示词 0.1：创建项目结构

```
# 任务：创建顺时后端项目骨架

## 技术栈
Python 3.11, FastAPI, SQLAlchemy 2.0, Alembic, Celery, Redis, PostgreSQL 16

## 要求
请创建以下文件的完整内容：

1. `backend/requirements.txt` — 包含所有依赖及版本号：
fastapi==0.109.0
uvicorn[standard]==0.27.0
sqlalchemy==2.0.25
alembic==1.13.1
asyncpg==0.29.0
psycopg2-binary==2.9.9
redis==5.0.1
celery==5.3.6
pydantic==2.5.3
pydantic-settings==2.1.0
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4
python-multipart==0.0.6
httpx==0.26.0
pillow==10.2.0
meilisearch==0.31.0
openai==1.12.0
dashscope==1.14.1
oss2==2.18.4
jpush==3.3.9

2. `backend/app/config.py` — 使用pydantic-settings的Settings类，包含所有环境变量：
DATABASE_URL, REDIS_URL, JWT_SECRET_KEY, JWT_ALGORITHM, JWT_ACCESS_TOKEN_EXPIRE_MINUTES,
QWEN_API_KEY, OPENAI_API_KEY, OSS_ACCESS_KEY_ID, OSS_ACCESS_KEY_SECRET, OSS_BUCKET_NAME,
MEILI_HOST, MEILI_MASTER_KEY, JPUSH_APP_KEY, JPUSH_MASTER_SECRET,
WECHAT_APP_ID, WECHAT_APP_SECRET, WECHAT_MCH_ID,
ALIPAY_APP_ID, ALIPAY_PRIVATE_KEY, APP_ENV, APP_URL

3. `backend/app/database.py` — SQLAlchemy异步引擎配置 + Redis连接 + get_db依赖
使用create_async_engine + async_sessionmaker

4. `backend/app/main.py` — FastAPI应用入口：
- CORS中间件
- 异常处理器
- 路由注册（先占位）
- 启动/关闭事件（数据库连接）
- 健康检查 /health 端点

5. `backend/Dockerfile` — Python 3.11 slim镜像

6. `backend/docker-compose.yml` — 包含api, celery_worker, celery_beat, db(pgvector), redis, meilisearch

## 输出要求
- 输出每个文件的完整内容，不省略
- 包含所有import
- 包含中文注释说明
- 确保可以直接 docker-compose up 运行
```

提示词 0.2：创建基础异常和响应模型

```
# 任务：创建基础异常处理和统一响应模型

## 项目上下文
顺时是AI养生App的后端，使用FastAPI。

## 已完成
- backend/app/config.py (配置类)
- backend/app/database.py (数据库连接)
- backend/app/main.py (入口)

## 本次要求

1. `backend/app/core/exceptions.py`
   - 自定义异常基类 AppException(status_code, code, message)
   - 具体异常: AuthenticationError, PermissionError, NotFoundError, ValidationError, RateLimitError, PaymentError
   - 全部message用温暖的中文，例如：
     - AuthenticationError: "登录已过期，请重新登录"
     - NotFoundError: "找不到你要的内容"
     - RateLimitError: "请求太频繁了，休息一下吧"

2. `backend/app/core/response.py`
   - 统一成功响应函数 success_response(data=None, message="success")
     → {"code": 0, "message": "success", "data": data}
   - 统一分页响应 paginated_response(items, total, cursor, limit)
     → {"code": 0, "data": {"items": [...], "total": N, "cursor": "xxx", "has_more": true}}

3. `backend/app/core/dependencies.py`
   - get_current_user 依赖 (从JWT token解析用户)
   - get_current_active_user (确保用户未被删除)
   - get_premium_user (确保是付费会员)
   - require_admin (确保是管理员)

## 输出要求
每个文件完整输出，包含所有import和类型注解。
```

第四章：Phase 1 — 数据库模型（精确提示词）

关键原则

- 每次只创建1-2个模型文件
- 每次都附上对应的SQL建表语句作为参考
- 要求AI同时创建对应的Pydantic Schema

提示词 1.1：用户模型

```
# 任务：创建用户数据模型

## 技术栈
SQLAlchemy 2.0 声明式映射, PostgreSQL 16, UUID主键

## 代码规范
- 使用 mapped_column 而非 Column
- 使用 Mapped[type] 类型注解
- UUID使用 postgresql.UUID(as_uuid=True)
- JSONB使用 postgresql.JSONB
- 时间使用 DateTime(timezone=True)
- 必须定义 __tablename__
- 每个字段加中文注释

## 请创建以下文件

### 1. backend/app/models/user.py

参考以下SQL设计（必须严格匹配字段）：

```sql
CREATE TABLE users (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 phone VARCHAR(20) UNIQUE,
 email VARCHAR(255) UNIQUE,
 password_hash VARCHAR(255),
 nickname VARCHAR(50),
 avatar_url VARCHAR(500),
 gender SMALLINT DEFAULT 0,
 birth_date DATE,
 province VARCHAR(50),
 city VARCHAR(50),
 constitution_type VARCHAR(20),
 constitution_tested_at TIMESTAMPTZ,
 constitution_scores JSONB,
 is_pregnant BOOLEAN DEFAULT FALSE,
 pregnancy_week SMALLINT,
 is_menstrual_tracking BOOLEAN DEFAULT FALSE,
 last_period_date DATE,
 avg_cycle_days SMALLINT DEFAULT 28,
 membership_level SMALLINT DEFAULT 0,
 membership_expires_at TIMESTAMPTZ,
 membership_auto_renew BOOLEAN DEFAULT FALSE,
 membership_platform VARCHAR(20),
 total_points INTEGER DEFAULT 0,
 user_level SMALLINT DEFAULT 1,
 consecutive_days INTEGER DEFAULT 0,
 preferred_wake_time TIME DEFAULT '07:00',
 preferred_sleep_time TIME DEFAULT '22:30',
 notification_enabled BOOLEAN DEFAULT TRUE,
 notification_settings JSONB DEFAULT '{}',
 dietary_restrictions JSONB DEFAULT '[]',
 font_size VARCHAR(10) DEFAULT 'normal',
 created_at TIMESTAMPTZ DEFAULT NOW(),
 updated_at TIMESTAMPTZ DEFAULT NOW(),
 last_login_at TIMESTAMPTZ,
 last_active_at TIMESTAMPTZ,
 onboarding_completed BOOLEAN DEFAULT FALSE,
 is_active BOOLEAN DEFAULT TRUE,
 is_deleted BOOLEAN DEFAULT FALSE,
 register_source VARCHAR(30),
 referrer_id UUID
);
```

包含以下methods: - is\_premium → bool（检查会员是否有效） - is\_membership\_expired → bool - display\_gender → str（返回“男” / “女” / “未知”）

## 2. backend/app/schemas/user.py

使用Pydantic v2, 创建以下Schema: - UserBase（基础字段） - UserCreate（注册时） - UserUpdate（更新时, 所有字段Optional） - UserProfile（返回给前端的完整profile） - UserBrief（简要信息, 用于列表展示） - UserSettings（设置相关字段）

## 输出要求

- 完整文件, 所有import
- 类型注解完整
- 中文注释



# 任务：创建节气数据模型

## 项目上下文

[粘贴PROJECT\_CONTEXT.md的内容]

## 已完成

- backend/app/models/user.py
- backend/app/schemas/user.py

## 请创建

### 1. backend/app/models/solar\_term.py

参考SQL：[粘贴终极开发文档中solar\_terms和solar\_term\_instances的建表SQL]

### 2. backend/app/schemas/solar\_term.py

- SolarTermBase
- SolarTermDetail（完整养生内容）
- SolarTermBrief（简要信息）
- SolarTermCalendar（日历视图）
- CurrentSolarTerm（当前节气+倒计时）

## 输出要求

完整文件，所有字段严格匹配SQL设计。

```
以此模式继续创建所有模型（每次1-2个表）：
- 提示词1.3: constitution_types + questions + test_records
- 提示词1.4: recipes
- 提示词1.5: tea_recipes
- 提示词1.6: acupoints
- 提示词1.7: exercise_tutorials
- 提示词1.8: articles + audio_contents
- 提示词1.9: chat_sessions + chat_messages
- 提示词1.10: wellness_journals + wellness_reports
- 提示词1.11: membership_orders + point_records + referral_codes + coupons
- 提示词1.12: banners + community_posts + challenges
- 提示词1.13: user_favorites + user_reminders + analytics_events
- 提示词1.14: knowledge_chunks (pgvector向量表)

每完成一个，更新PROJECT_CONTEXT.md的"已完成文件"列表。

提示词 1.15 : Alembic迁移
```



# 任务：创建数据库迁移

## 已完成

所有Model文件已创建完毕（列出全部路径）

## 要求

1. 初始化Alembic配置：`backend/alembic.ini` 和 `backend/migrations/env.py`
2. 在env.py中正确导入所有Model
3. 创建初始迁移：包含全部表
4. 确保UUID扩展和pgvector扩展在迁移中创建

## 输出

完整的alembic.ini和env.py，以及运行迁移的命令说明。

```

第五章：Phase 2 - Service层（精确提示词模板）

关键原则
- **每个Service文件一个提示词**
- **Service层包含全部业务逻辑，API层只做路由和参数校验**
- **每个Service函数都要有完整的docstring**

提示词 2.1：节气引擎服务
```

# 任务：创建节气计算引擎服务

---

## 项目上下文

[PROJECT\_CONTEXT.md]

## 已完成

- 所有Model和Schema文件

## 本次要求

创建 `backend/app/services/solar_term_engine.py`

包含以下函数（每个都要完整实现，不要省略）：

1. `async def get_current_solar_term(db: AsyncSession) -> SolarTermDetail`
  - 查询`solar_term_instances`表，找到当前年份中最近的已过节气
  - 返回该节气的完整信息
  - 计算距下一个节气的天数
  - 结果缓存到Redis（key: “global:solar\_term:current”，TTL: 3600s）
2. `async def get_upcoming_solar_term(db: AsyncSession) -> dict`
  - 返回下一个即将到来的节气
  - 包含倒计时天数
3. `async def get_solar_term_calendar(db: AsyncSession, year: int) -> list`
  - 返回指定年份的全部24个节气日期
4. `async def get_solar_term_by_code(db: AsyncSession, code: str) -> SolarTermDetail`
  - 按code查询节气完整信息
5. `async def get_solar_term_recommendations(db: AsyncSession, code: str, constitution_type: str) -> dict`
  - 交叉查询节气×体质的个性化建议
  - 返回食谱、茶饮、穴位推荐
6. `def check_solar_term_change() -> bool`
  - 检查今天是否是新节气（供Celery定时任务调用）

## 技术要求

- 使用Redis缓存（从`database.py`导入`redis_client`）
- 所有数据库查询用`async/await`
- 异常处理完整（查不到数据时抛`NotFoundError`）
- 包含logging

## 输出

完整文件，约150-250行代码。不要省略任何函数实现。



# 任务：创建体质评分服务

请创建 `backend/app/services/constitution_scorer.py`

包含以下函数：

1. `async def get_questions(db) -> list[ConstitutionQuestion]`
  - 返回全部33道题目，按sequence排序
2. `def calculate_scores(answers: dict[str, int]) -> dict`
  - 实现王琦标准评分算法： $\text{transformed\_score} = ((\text{raw\_score} - n) / (n * 4)) * 100$
  - 判定标准：平和质 $\geq 60$ 且其余 $< 40$ 为”是”；偏颇体质 $\geq 40$ 为”是”
  - 返回 `{scores: {各体质分数}, primary_type, secondary_type, judgments}`

具体的题目分组：

  - 平和质: `question_group= 'pinghe'` 的题目
  - 气虚质: `question_group= 'qixu'` 的题目
  - ... 以此类推
3. `async def submit_test(db, user_id, answers) -> ConstitutionTestResult`
  - 调用`calculate_scores`计算
  - 保存测试记录到数据库
  - 更新用户表的`constitution_type`
  - 触发AI报告生成（异步Celery任务）
  - 返回测试结果
4. `async def generate_ai_report(user_id, primary_type, secondary_type, scores) -> str`
  - 调用AI生成个性化体质报告
  - Prompt模板：[提供完整prompt]
  - 保存到`test_record`的`ai_report`字段

评分算法必须严格按照王琦标准实现，不能简化。

输出完整文件，不省略。

```

以此模式继续创建所有Service（每次1个文件）：
- 提示词2.3: ai_companion.py (AI对话核心 - System Prompt构建 + RAG + 流式响应)
- 提示词2.4: rag_retriever.py (知识检索)
- 提示词2.5: recommendation.py (个性化推荐引擎)
- 提示词2.6: journal_analyzer.py (日记AI分析)
- 提示词2.7: report_generator.py (周报/月报生成)
- 提示词2.8: notification_engine.py (推送引擎)
- 提示词2.9: payment_service.py (支付处理)
- 提示词2.10: share_card.py (分享海报生成)
- 提示词2.11: cache_manager.py (缓存管理)
- 提示词2.12: weather_service.py (天气养生)
- 提示词2.13: search_service.py (Meilisearch集成)

...

第六章：Phase 3 - API层（精确提示词模板）

关键原则
- **API路由函数只做：参数校验 -> 调用Service -> 返回响应**
- **不在API层写业务逻辑**
- **每个路由文件一个提示词**

提示词 3.1: 认证API
```

# 任务：创建认证API

---

## 项目上下文

[PROJECT\_CONTEXT.md]

## 已完成的Service

- backend/app/services/auth\_service.py（如果已创建） 否则请同时创建service

## 请创建 `backend/app/api/v1/auth.py`

包含以下端点（使用APIRouter）：

1. POST /auth/sms/send
  - Body: {phone: str}
  - 频率限制：1次/分钟/手机号，5次/小时
  - 生成6位验证码 → 存Redis（key: sms:{phone}, TTL: 300s）
  - 调用阿里云短信API发送（先打印到日志，后续替换）
  - 返回 { “code” : 0, “message” : “验证码已发送” }
2. POST /auth/sms/verify
  - Body: {phone: str, code: str}
  - 从Redis取验证码对比
  - 匹配 → 查找或创建用户 → 生成JWT tokens
  - 返回 {access\_token, refresh\_token, is\_new\_user, user: UserProfile}
3. POST /auth/wechat
  - Body: {code: str}
  - 用code换取access\_token和openid（调用微信API）
  - 查找或创建用户
  - 返回 {access\_token, refresh\_token, is\_new\_user, user}
4. POST /auth/apple
  - Body: {identity\_token: str, authorization\_code: str}
  - 验证identity\_token
  - 返回同上
5. POST /auth/refresh
  - Body: {refresh\_token: str}
  - 验证refresh\_token → 生成新的access\_token
  - 返回 {access\_token}
6. POST /auth/logout
  - Header: Authorization
  - 将refresh\_token加入Redis黑名单
  - 返回 { “code” : 0, “message” : “已退出登录” }

## JWT配置

- access\_token: 24小时过期
- refresh\_token: 30天过期
- 算法: HS256

## 输出

完整的auth.py文件 + 如果需要的auth\_service.py。 包含所有错误处理（验证码错误、过期、手机号格式错误等）。 所有错误消息用温暖的中文。

```
以此模式继续创建所有API路由文件，每次1个文件。按顺序：
- 3.2: user.py (用户信息+设置+家庭)
- 3.3: home.py (首页聚合接口 - 最复杂，调用多个Service)
- 3.4: solar_term.py
- 3.5: constitution.py
- 3.6: recipe.py
- 3.7: tea.py
- 3.8: acupoint.py
- 3.9: exercise.py
- 3.10: article.py + audio.py
- 3.11: chat.py (AI对话 - SSE流式响应)
- 3.12: journal.py + report.py
- 3.13: membership.py + payment.py
- 3.14: search.py
- 3.15: community.py + challenge.py
- 3.16: share.py + notification.py

第七章：Phase 4 - AI系统（最关键，需要最精确的提示词）

提示词 4.1：AI对话核心
```

# 任务：创建AI养生对话核心服务

这是整个项目最核心的模块，请特别认真对待。

请创建 `backend/app/services/ai_companion.py`

功能要求（按顺序实现）：

函数1: `build_system_prompt(user, solar_term, shichen, weather, rag_context) -> str`

构建动态System Prompt，必须包含以下所有变量注入： - 用户体质类型和对应的饮食原则 - 当前节气和养生要点 - 当前时辰和脏腑建议 - 天气信息 - 是否孕期（如果是，加入孕期安全规则） - 是否经期 - 饮食限制 - RAG检索到的知识片段

完整的System Prompt模板如下（必须完整使用，不要简化）： [粘贴终极开发文档第五部分5.1的完整System Prompt]

函数2: `async def retrieve_knowledge(query: str, top_k: int = 5) -> str`

RAG知识检索： - 将用户问题转为embedding（调用通义embedding API） - 在knowledge\_chunks表中做向量相似度搜索 - 返回top\_k个最相关的知识片段 - 格式化为context字符串

函数3: `async def chat_stream(session_id, user_id, user_message, db) -> AsyncGenerator`

流式对话核心： 1. 检查用户今日对话次数（免费3次/天，会员无限） 2. 保存用户消息到数据库 3. 获取历史消息（最近10条） 4. 获取用户信息、当前节气、时辰、天气 5. 调用RAG检索相关知识 6. 构建System Prompt 7. 调用AI API（流式输出） 8. 逐chunk yield SSE格式数据 9. 流式结束后保存完整AI回复 10. 解析回复中的推荐卡片标签 11. yield推荐卡片数据 12. yield [DONE]

主模型：通义千问qwen-max（会员）/ qwen-turbo（免费） 备用模型：gpt-4o-mini 自动降级策略：主模型超时30s或报错 → 切换备用

函数4: `def parse_recommendation_cards(text: str) -> list[dict]`

解析AI回复中的 [推荐:类型:ID:名称] 标签 返回 [{type, id, title, action: “navigate” }]

## 代码质量要求

- 完整的异常处理（AI超时、API错误、余额不足等）
- 详细的logging（记录每次请求的token用量、响应时间）
- 类型注解完整
- 约300-400行代码

请输出完整文件，不要省略任何实现。

```

第八章：Phase 6-7 — 前端开发（提示词策略）

前端开发特殊策略

前端开发更需要“逐组件”的方式。策略：
```

1. 先创建设计系统组件（Design System）
2. 再创建Store（状态管理）
3. 再创建API Service
4. 最后逐页面开发

### 提示词 6.1：设计系统



# 任务：创建顺时App设计系统基础组件

## 设计规范

品牌色：#2D5A3D（墨绿）、#E8C4A0（暖杏）、#F7F3EE（雅白）、#C9A96E（浅金） 字体：系统默认 圆角：8dp（小）、12dp（中）、16dp（大） 间距基数：4dp（4/8/12/16/20/24/32）

## 请创建以下组件

### 1. mobile/src/theme/colors.ts

导出所有品牌色 + 语义色（success/warning/error/info）

### 2. mobile/src/theme/spacing.ts

导出间距常量

### 3. mobile/src/components/ui/Button.tsx

- 变体：primary（墨绿底白字） / secondary（描边） / ghost（纯文字）
- 尺寸：small / medium / large
- 支持loading状态
- 支持disabled状态
- 使用StyleSheet，不用Tailwind

### 4. mobile/src/components/ui/Card.tsx

- 白色圆角卡片，带阴影
- 支持图片+标题+副标题布局

### 5. mobile/src/components/ui/Input.tsx

- 标签+输入框+错误提示
- 支持password模式

### 6. mobile/src/components/ui/Tag.tsx

- 小标签组件（体质标签/功效标签/节气标签）
- 支持多种颜色

所有组件使用TypeScript + React Native，包含完整类型定义和Props接口。

# 任务：开发顺时App首页

## 项目上下文

[PROJECT\_CONTEXT.md + 已完成的组件列表]

## 首页数据来源

调用 GET /api/v1/home 接口，返回：{ solar\_term: {name\_cn, illustration\_url, wellness\_principle, days\_to\_next}, shichen: {name, organ, advice}, daily\_tips: {diet, tea, acupoint, exercise}, weather\_tips: [{advice, diet}], checkin\_status: {completed: [], pending: [], streak\_days: N}, recommended\_recipes: [{id, title, cover\_image\_url, functions}], recommended\_articles: [{id, title, cover\_image\_url, excerpt}], recommended\_audio: [{id, title, cover\_image\_url, duration}], daily\_quote: {text, source} }

## 页面结构（从上到下，严格按此顺序）

- 顶部问候栏
  - “早安/午安/晚安, {昵称}”
  - 右侧通知铃铛
- 节气英雄卡片（占满宽度，高200dp）
  - 背景：节气插画
  - 左侧：节气名（大字30sp）+ 养生一句话
  - 右侧：距下一节气{N}天
  - 点击跳转节气详情页
- 今日养生建议（横向FlatList，4张卡片）
  - 每张卡片：图标 + 标题 + 一句话建议
  - 点击展开详情
- 天气提示（条件渲染，有天气数据时显示）
  - 黄色提示条
- 今日打卡区域
  - 横向显示可打卡项（checkbox风格）
  - 底部：连续{N}天打卡
- 快捷入口（横向6个圆形图标）
- 推荐食谱（横向卡片FlatList）
- 推荐阅读（纵向列表，2-3条）
- 每日名言

## 技术要求

- 使用ScrollView包裹
- 使用Zustand Store获取数据
- 下拉刷新
- 骨架屏加载态
- 完整的TypeScript类型

请输出完整的 HomeScreen.tsx 文件和相关子组件。

---  
## 第九章：质量保证检查清单  
### 每个文件完成后的检查项

☐ 文件可以独立运行（无语法错误） ☐ 所有import语句正确且存在 ☐ 没有 “...” 或 “# TODO” 占位（除非是明确计划后续实现的） ☐ 包含完整的错误处理（try/except） ☐ 包含类型注解 ☐ API响应格式统一 ☐ 错误消息使用中文 ☐ 数据库查询使用参数化（无SQL注入风险） ☐ 异步函数使用async/await ☐ 敏感信息不硬编码 ☐ 与已完成的文件接口一致（函数签名/返回值/类型）

### 每个Phase完成后的集成检查

Phase 1完成后： ☐ 运行 alembic upgrade head 成功 ☐ 所有表创建成功 ☐ 可以手动插入测试数据

Phase 2完成后： ☐ 每个Service的核心函数可以单独调用 ☐ Redis连接正常 ☐ AI API调用测试通过

Phase 3完成后： ☐ 所有API端点可以通过 /docs 页面访问 ☐ Swagger文档显示正常 ☐ 用Postman/httpx测试核心接口

Phase 4完成后： ☐ AI对话流式响应正常 ☐ RAG检索返回相关结果 ☐ 推荐卡片正确解析

Phase 7完成后： ☐ App可以在模拟器中运行 ☐ 首页正常加载数据 ☐ 导航流程正确

---  
## 第十章：常见问题和修复提示词模板  
### 当AI生成的代码有bug时

# 修复任务

---

## 当前文件

[粘贴完整的当前文件内容]

## 错误信息

[粘贴完整的错误信息/traceback]

## 问题描述

[描述什么行为不符合预期]

## 要求

- 找出错误原因
- 提供修复后的完整文件（不要只给diff，给完整文件）
- 解释为什么这样修复

### 当AI生成的代码不完整时

# 补全任务

## 当前文件

[粘贴当前文件]

## 缺失部分

以下函数只有签名没有实现，请完整实现： 1. [函数名1] — 应该做什么 2. [函数名2] — 应该做什么

## 重要

不要输出整个文件，只输出需要替换的函数的完整实现。但每个函数必须完整，包含所有逻辑和错误处理。

### 当需要重构/优化时

# 重构任务

## 当前文件

[粘贴完整文件]

## 问题

[描述具体问题：性能差/代码重复/结构不合理/...]

## 要求

- 保持所有现有功能不变
- 优化[具体方面]
- 输出完整的重构后文件
- 说明改了什么，为什么

## 不允许

- 不要改变函数签名（除非问题就出在签名上）
- 不要删除任何功能
- 不要改变API响应格式

```

第十一章：提示词增强技巧（通用）

技巧1：角色设定

每次对话开头加入：
```

你是一位有10年经验的Python/FastAPI高级工程师，擅长构建生产级API系统。你写的代码注重：1. 类型安全 2. 完整的错误处理 3. 清晰的代码结构 4. 详细的日志记录 5. 性能优化 请按照以上标准完成以下任务。

```
技巧2：Few-Shot示例

当AI不理解你要的代码风格时，先给一个例子：
```

## 代码风格示例

以下是本项目中已有的一个Service函数示例，请严格按照相同风格编写新代码：

```
async def get_recipe_by_id(db: AsyncSession, recipe_id: UUID) -> RecipeDetail:
 """根据ID获取食谱详情。"""
 result = await db.execute(
 select(Recipe).where(Recipe.id == recipe_id, Recipe.is_published == True)
)
 recipe = result.scalar_one_or_none()
 if not recipe:
 raise NotFoundError("找不到这个食谱")

 # 更新浏览计数
 recipe.view_count += 1
 await db.commit()

 logger.info(f"Recipe viewed: {recipe_id}")
 return RecipeDetail.model_validate(recipe)
```

请按照相同的风格编写下面的函数…

### 技巧3：分步输出  
当一个文件很长（>300行）时：

这个文件较长，请分两部分输出： 第一部分：类定义和辅助函数（约1-150行） 第二部分：核心业务函数（约151-300行）

先输出第一部分。输出完成后我会说” 继续”，你再输出第二部分。 不要在第一部分用” … ”省略任何内容。

### 技巧4：验证+迭代

请先不写代码，先回答以下问题确认你的理解： 1. 这个Service需要注入哪些依赖？ 2. 主要的数据库查询有哪几个？ 3. 哪些操作需要事务？ 4. 错误场景有哪些？ 5. 缓存策略是什么？

确认理解后我再让你写代码。

### 技巧5：强制完整输出

⚠ 重要提示： - 不要使用 “…” 省略任何代码 - 不要写 “// 其他代码类似” - 不要写 “// 同上” - 必须输出完整的、可以直接复制运行的文件 - 如果文件太长，主动说明并分段输出，但每段都必须完整

```

第十二章：SEASONS海外版开发补充说明

SEASONS海外版使用完全相同的开发策略，但提示词中需要替换以下内容：

| 替换项 | 顺时 | SEASONS |
|-----|-----|-----|
| 项目描述 | AI养生生活陪伴系统 | AI Calm Lifestyle Companion |
| 内容体系 | 二十四节气 | 四季十二月 + 南北半球 |
| 体质体系 | 九种体质/33题 | 七种Body Type/12题 |
| 数据库模型 | solar_terms | seasons |
| 数据库模型 | constitution_types | body_types |
| 用户字段 | phone | email |
| 认证方式 | 手机号+微信 | Email+Google+Apple |
| 支付 | 微信/支付宝 | Stripe |
| 推送 | 极光推送 | Firebase FCM |
| 云服务 | 阿里云 | AWS |
| AI | 通义千问 | GPT-4o |
| 语言 | 中文 | 英文（+多语言） |
| 合规 | 个保法 | GDPR+CCPA |

SEASONS特有的额外提示词

半球检测服务：
```

# 任务：创建南北半球季节计算服务

---

SEASONS的核心差异是支持南北半球。南半球（澳大利亚、新西兰、南美等）的季节与北半球相反： - 北半球3月=春天，南半球3月=秋天 - 必须根据用户的hemisphere字段自动翻转

请创建 hemisphere\_engine.py ...

**\*\*Stripe集成:\*\***



# 任务：创建Stripe订阅管理服务

包含： 1. 创建Stripe客户 2. 创建Checkout Session 3. 处理5种Webhook事件 4. 订阅状态管理 5. 退款处理 ...

\*\*GDPR合规：\*\*

# 任务：创建GDPR合规API

包含：1. 数据导出API（生成用户全部数据的JSON） 2. 账号删除API（匿名化+30天后永久删除） 3. 同意管理API 4. 数据可移植性 ...

---

## 附录A：完整的开发任务清单（Checklist）

Phase 0: 骨架 (6个提示词) ☐ 0.1 项目结构 + Docker ☐ 0.2 异常处理 + 响应模型 ☐ 0.3 认证基础（JWT工具函数） ☐ 0.4 Redis 工具函数 ☐ 0.5 OSS上传工具函数 ☐ 0.6 项目记忆文件初始化

Phase 1: 数据库 (15个提示词) ☐ 1.1 User Model + Schema ☐ 1.2 SolarTerm Model + Schema ☐ 1.3 Constitution Models + Schema ☐ 1.4 Recipe Model + Schema ☐ 1.5 Tea Model + Schema ☐ 1.6 Acupoint Model + Schema ☐ 1.7 Exercise Model + Schema ☐ 1.8 Article + Audio Models ☐ 1.9 Chat Models ☐ 1.10 Journal + Report Models ☐ 1.11 Membership + Payment Models ☐ 1.12 Community + Challenge Models ☐ 1.13 Notification + Analytics Models ☐ 1.14 Knowledge Vector Table ☐ 1.15 Alembic Migration

Phase 2: Service层 (13个提示词) ☐ 2.1 solar\_term\_engine.py ☐ 2.2 constitution\_scorer.py ☐ 2.3 ai\_companion.py ☐ 2.4 rag\_retriever.py ☐ 2.5 recommendation.py ☐ 2.6 journal\_analyzer.py ☐ 2.7 report\_generator.py ☐ 2.8 notification\_engine.py ☐ 2.9 payment\_service.py ☐ 2.10 share\_card.py ☐ 2.11 cache\_manager.py ☐ 2.12 weather\_service.py ☐ 2.13 search\_service.py

Phase 3: API层 (16个提示词) ☐ 3.1 auth.py ☐ 3.2 user.py ☐ 3.3 home.py ☐ 3.4 solar\_term.py ☐ 3.5 constitution.py ☐ 3.6 recipe.py ☐ 3.7 tea.py ☐ 3.8 acupoint.py ☐ 3.9 exercise.py ☐ 3.10 article.py + audio.py ☐ 3.11 chat.py (SSE) ☐ 3.12 journal.py + report.py ☐ 3.13 membership.py + payment.py ☐ 3.14 search.py ☐ 3.15 community.py + challenge.py ☐ 3.16 share.py + notification.py

Phase 4: AI系统 (4个提示词) ☐ 4.1 AI对话核心 ☐ 4.2 知识库向量化脚本 ☐ 4.3 知识库数据导入脚本 ☐ 4.4 AI回复质量测试

Phase 5: Celery (3个提示词) ☐ 5.1 定时任务配置 ☐ 5.2 异步任务实现 ☐ 5.3 Celery worker配置

Phase 6: 前端基础 (5个提示词) ☐ 6.1 设计系统组件 ☐ 6.2 导航结构 ☐ 6.3 Store定义 ☐ 6.4 API Service ☐ 6.5 认证流程

Phase 7: 前端页面 (12个提示词) ☐ 7.1 首页 ☐ 7.2 节气页 ☐ 7.3 食谱列表+详情 ☐ 7.4 茶饮列表+详情 ☐ 7.5 穴位列表+详情 ☐ 7.6 功法教程+跟练 ☐ 7.7 AI对话页 ☐ 7.8 日记页+日历 ☐ 7.9 报告页 ☐ 7.10 个人中心 ☐ 7.11 会员+支付 ☐ 7.12 引导流程

Phase 8: 集成 (4个提示词) ☐ 8.1 端到端测试 ☐ 8.2 性能优化 ☐ 8.3 安全审查 ☐ 8.4 Bug修复

Phase 9: 部署 (3个提示词) ☐ 9.1 生产Docker配置 ☐ 9.2 CI/CD配置 ☐ 9.3 监控告警配置

总计：81个提示词，产出150个代码文件 ``

文档版本：V1.0 用途：规范AI编程工具使用流程，确保开发质量 适用工具：GLM 5.1 / Cursor / Claude Code / Copilot / 通义灵码